

Web Application Framework: Django

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the model-template-view (MTV) architectural pattern and provides a robust set of tools for building web applications.

Create a Django project:

```
django-admin startproject blogproject
```

- This will create a project structure something like:

```
blogproject/  
manage.py  
blogproject/  
__init__.py  
settings.py  
urls.py  
wsgi.py
```

Create a Django app:

```
python manage.py startapp blog
```

provide the app structure of this command: `python manage.py startapp blog`

```
blog/  
init.py  
admin.py  
apps.py  
models.py  
tests.py  
views.py  
migrations/  
init.py
```

define views in views.py:

```
from django.shortcuts import render
from django.http import HttpResponse
#dummy data for testing
posts = [
    {
        'title': 'First Post',
        'content': 'This is the content of the first post.',
        'author': 'John Doe',
        'date_posted': 'June 1, 2024'
    },
    {
        'title': 'Second Post',
        'content': 'This is the content of the second post.',
        'author': 'Jane Smith',
        'date_posted': 'June 2, 2024'
    }
]
# Create your views here.
def home(request):
    context = {
        'posts': posts
    }
    return render(request, 'blog/home.html', context)

def about(request):
    return render(request, 'blog/about.html', {'title': 'About'})
```

Database in Django

Django uses:

SQLite (default)

☑ Already configured

☑ No installation needed

- settings.py:

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

- 'db.sqlite3' - This file will be auto-created after migrations are run. It will contain the database for your project.

Define models in models.py:

```
from django.db import models
```

```
# Create your models here.
```

```
class Post(models.Model):  
    title = models.CharField(max_length=100)  
    content = models.TextField()  
    author = models.CharField(max_length=50)  
    date_posted = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.title
```

how many different datatypes are there in the above model?

- there are 4 different datatypes in the above model:

1. CharField: used for the title and author fields, which are short text fields.
2. TextField: used for the content field, which is a longer text field.
3. DateTimeField: used for the date_posted field, which stores date and time information.
4. AutoField: used for the id field, which is an automatically generated primary key for each post (not explicitly defined in the model but automatically added by Django).

Migrations:

- Migrations are a way to propagate changes you make to your models (adding a field, deleting a model, etc.) into your database schema. They are like version control for your database schema. When you make changes to your models, you create a migration file that describes those changes, and then you apply the migration to update the database schema accordingly.
- To create a migration after defining your model, you run:

```
python manage.py makemigrations
```

To apply migrations:

```
python manage.py migrate
```

- This will create the necessary tables in the database based on your model definitions.

Admin Panel

```
from django.contrib import admin  
from .models import Post
```

```
admin.site.register(Post)
```

The above code registers the Post model with the Django admin site.

By doing this, you can manage the Post model through the Django admin interface.

This means you can create, read, update, and delete instances of the Post model directly from the admin dashboard without needing to write any additional code for handling these operations.

where to find this admin interface?

- You can access the Django admin interface by running your development server and navigating to
 - `http://localhost:8000/admin/` in your web browser.
 - You will need to create a superuser account to log in to the admin interface.
 - You can create a superuser by running the following command in your terminal:
 - Follow the prompts to enter a username, email, and password for the superuser account.
 - Once you have created the superuser account, you can log in to the admin interface using the credentials you just created.
 - After logging in, you will see the Post model listed under the "Blog" section (or whatever you named your app).
 - You can click on the Post model to view, add, edit, or delete posts through the admin interface.
-

Registering the app:

- After creating your app, you need to register it in your project's settings.py file:

```
INSTALLED_APPS = [  
    # other apps  
    'blog.apps.BlogConfig',  
]
```

'blog.apps.BlogConfig' is the path to the app's configuration class, which is defined in the apps.py file of your app.

This tells Django to include your app in the project and allows you to use its features, such as models, views, and templates.

Defining templates:

- In your Django app, you typically create a templates directory to store your HTML templates.
- Inside the templates directory, you can create a subdirectory with the same name as your app (e.g., blog) to organize your templates.
- For example, you would create the following directory structure:

```
blog/  
    templates/  
        blog/  
            base.html  
            home.html  
            about.html
```

- In your views, you can then render these templates by specifying the path to the template file. For example, in the home view, you would use:
- This tells Django to look for the home.html template in the blog/templates/blog/ directory.

base.html:

```
{% load static %}  
<!DOCTYPE html>  
<html>  
<head>
```

```

<!-- Required meta tags -->
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

<!-- Bootstrap CSS -->
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/
bootstrap.min.css" integrity="sha384-
Gn5384xqQ1aoWXA+058RXPxPg6fy4IWvTNh0E263XmFcJlSAwiGgFAW/dAiS6JXm"
crossorigin="anonymous">

<link rel="stylesheet" type="text/css" href="{% static 'blog/main.css' %}">

{% if title %}
    <title>Django Blog - {{ title }}</title>
{% else %}
    <title>Django Blog</title>
{% endif %}
</head>
<body>
<!--Navbar-->
{% block content %}{% endblock %}
<!--Sidebar-->

```

home.html:

```

{% extends "blog/base.html" %}
{% block content %}
    {% for post in posts %}
        <article class="media content-section">
            <div class="media-body">
                <div class="article-metadata">
                    <a class="mr-2" href="#">{{ post.author }}</a>
                    <small class="text-muted">{{ post.date_posted }}</small>
                </div>
                <h2><a class="article-title" href="#">{{ post.title }}</a></h2>
                <p class="article-content">{{ post.content }}</p>
            </div>
        </article>
    {% endfor %}
{% endblock content %}

```

defining url patterns:

-- In your project's urls.py file, you include the URL patterns from your app.

- For example, in the blogproject/urls.py file, you would have:

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    # path('admin/', admin.site.urls),
    path("", include('blog.urls')),
]
```

- In your app's urls.py file, you define URL patterns that map to your views.
- For example, in the blog/urls.py file, you might have the following URL patterns:

```
from django.urls import path
from . import views
urlpatterns = [
    path("", views.home, name='blog-home'),
    path('about/', views.about, name='blog-about'),
]
```

To run the development server, use the following command:

- This will start the Django development server, and you can access your application at <http://localhost:8000/> in your web browser.
 - You should see the home page of your blog application, and you can navigate to the about page by going to <http://localhost:8000/about/>.
 - You can also access the admin interface at <http://localhost:8000/admin/> to manage your posts and other models.
-

Designing a RESTful Web API:

- REST (Representational State Transfer) is an architectural style for designing networked applications.
 - A RESTful API is an API that adheres to the principles of REST.
 - RESTful APIs use HTTP methods (GET, POST, PUT, DELETE) to perform CRUD operations on resources.
 - Resources are identified by URLs (Uniform Resource Locators).
 - RESTful APIs are stateless, meaning that each request from a client to the server must contain all the information needed to understand and process the request.
 - RESTful APIs can return data in various formats, such as JSON (JavaScript Object Notation) or XML (eXtensible Markup Language).
 - RESTful APIs are widely used in web development to enable communication between different systems and applications.
 - They allow developers to create scalable and flexible APIs that can be easily consumed by client.
-

- To create a RESTful API in Django, you can use the Django REST framework, which provides a powerful and flexible toolkit for building Web APIs.
- `django-rest-framework` is a third-party package that you can install using pip:

pip `install` `django-rest-framework`

Add to `settings.py`:

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
]
```

Step 1: Create the new app

In your terminal, within the `blogproject` directory:

```
python manage.py startapp stations
```

Step 2: Register the app

In blogproject/settings.py, add 'stations' and 'rest_framework' (if not already there):

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
    'stations',  
]
```

Step 3: The Model (stations/models.py)

We'll use the textbook's station structure but use proper field types for 2026.

```
from django.db import models  
  
class Station(models.Model):  
    name = models.CharField(max_length=100)  
  
    # Using DateTimeField is more modern than CharField for timestamps  
    timestamp = models.DateTimeField(auto_now_add=True)  
  
    temperature = models.FloatField()  
  
    lat = models.DecimalField(max_digits=9, decimal_places=6)  
  
    lon = models.DecimalField(max_digits=9, decimal_places=6)  
  
    def __str__(self):  
        return self.name
```

Step 4: The Serializer with Validation (stations/serializers.py)

Here we implement Requirement C: Logic to ensure data integrity.

```
from rest_framework import serializers  
from .models import Station
```

```

class StationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Station
        fields = '__all__'

# C. Validation Logic: Ensure temperature is within a realistic range
def validate_temperature(self, value):
    if value < -50 or value > 60:
        raise serializers.ValidationError("Temperature out of valid sensor range.")
    return value

# The StationSerializer is a ModelSerializer that automatically generates fields based on the
Station model.
# The Meta class specifies that the serializer should use the Station model and include all
fields.
# The validate_temperature method is a custom validation method that checks if the
temperature value is within
# a realistic range for weather stations. If the value is less than -50 or greater than 60,
# it raises a ValidationError with a message indicating that the temperature is out of valid
sensor range.
# If the value is valid, it simply returns the value.
# This ensures that any data being saved to the database through
# this serializer meets the specified criteria for temperature.

```

Step 5: The ViewSet with Logic & Actions (stations/ views.py)

Here we implement Requirement A (Method Overriding) and Requirement B (Custom Actions).

```

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response
from .models import Station
from .serializers import StationSerializer

class StationViewSet(viewsets.ModelViewSet):
    queryset = Station.objects.all()
    serializer_class = StationSerializer

```

```
# A. Overriding Default Methods: perform_create
# Useful for logging or adding data the user didn't provide
def perform_create(self, serializer):
    print(f"Adding new station data for: {self.request.data.get('name')}")
    serializer.save()
```

```
# B. Using the @action Decorator: Custom endpoint
# Access this at: /api/stations/all_names/
@action(detail=False, methods=['get'])
def all_names(self, request):
    names = Station.objects.values_list('name', flat=True)
    return Response(names)
```

```
# Access this at: /api/stations/1/ping/
@action(detail=True, methods=['get'])
def ping(self, request, pk=None):
    station = self.get_object()
    return Response({'status': f'Station {station.name} is active and reporting.'})
```

```
# what does detail=True do? It means this action is for a specific instance,
# so it will require a primary key (pk) in the URL to identify which station to ping.
```

```
# how does django know that the ping method is for /api/stations/1/ping/ this api only?
# Because of the @action decorator with detail=True,
# Django REST Framework automatically generates a URL pattern for this method that
includes the primary key (pk) of the station instance.
# So when you access /api/stations/1/ping/, it will call the ping method for the station with
pk=1.
# then lets say if I access /api/stations/1/ponng what will happen?
# If you access /api/stations/1/ponng, you will get a 404 Not Found error because there is no
URL pattern defined for "ponng".
# The @action decorator only creates a URL pattern for the "ping" method, so any other URL
that does not match the defined patterns will result in a 404 error.
# what if I access /api/stations/1/pingg?
# If you access /api/stations/1/pingg, you will also get a 404 Not Found error for the same
reason as above.
# The URL pattern for the "ping" method is specifically defined as /api/stations/<pk>/ping/,
# so the url pattern is created based on method name, like the ping method will create a url
pattern with ping in it,
# so if you change the method name to pingg then the url pattern will also change to /api/
stations/<pk>/pingg/ and
# if you try to access /api/stations/1/pingg/ then it will work
# but if you try to access /api/stations/1/ping/ then it will give you a 404 error because there is
```

no url pattern for ping anymore.

Step 6: App URLs (stations/urls.py)

The router automatically creates all CRUD routes for you.

```
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from .views import StationViewSet
```

```
router = DefaultRouter()
router.register(r'stations', StationViewSet)
```

```
urlpatterns = [
    path("", include(router.urls)),
]
```

Step 7: Project URLs (blogproject/urls.py)

Update your main configuration to include the new API.

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('blog/', include('blog.urls')),
    path('api/', include('stations.urls')), # Our new REST API path
]
```

Step 8: Templates & Static Files

While REST APIs primarily serve JSON, you can still serve a dashboard.

The File: stations/templates/stations/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Station Dashboard</title>
</head>
<body>
  <h1>IoT Station Monitor</h1>
  <p>The API is live at <a href="/api/stations/">/api/stations/</a></p>

  <script>
    fetch('/api/stations/')
      .then(response => response.json())
      .then(data => console.log("Current Station Data:", data));
  </script>
</body>
</html>
```

To serve this template, you would add a simple functional view in views.py and a path('dashboard/', views.dashboard) in urls.py.

Execution Steps

Migrations:

```
python manage.py makemigrations
python manage.py migrate
```

Run Server:

```
python manage.py runserver
```

Test it:

- Go to <http://127.0.0.1:8000/api/stations/> to see the Browsable API.
- Try creating a station with a temperature of 100 to see your Validation Logic in action.

- Go to <http://127.0.0.1:8000/api/stations/1/ping/> (if you have an object with ID 1) to see your @action working.

This structure keeps your code modular. The Logic is in the Serializer and ViewSet, the Data is in the Model, and the Routing is automated by the Router.

testing api endpoints

1. List of API Endpoints

Action	HTTP Method	URL Path	Description
List	GET	/api/stations/	Returns a list of all stations in JSON
Create	POST	/api/stations/	Adds a new station (Triggers perform_create)
Retrieve	GET	/api/stations/{id}/	Returns details of one specific station
Update	PUT	/api/stations/{id}/	Replaces all fields of a specific station
Partial Update	PATCH	/api/stations/{id}/	Updates only specific fields of a station
Delete	DELETE	/api/stations/{id}/	Removes the station record
All Names	GET	/api/stations/all_names/	Custom action returning only a list of names
Ping	GET	/api/stations/{id}/ping/	Custom action returning a status message

2. Test Cases and Expected Results

You can test these using the Django REST Framework Browsable API or Postman/curl.

Case 1: Creating a Valid Station (Testing Logic A)

- Method: POST
- URL: <http://127.0.0.1:8000/api/stations/>

```
{  
  "name": "UVCE_Main_Lab",  
  "temperature": 24.5,  
  "lat": 12.9716,  
  "lon": 77.5946  
}
```

Expected Result:

- Status Code: 201 Created
 - Response Body: JSON object with id and timestamp
 - Console Output: perform_create print message
-

Case 2: Validation Failure (Testing Logic C)

- Method: POST
- URL: <http://127.0.0.1:8000/api/stations/>

```
{  
  "name": "Extreme_Sensor",  
  "temperature": 150.0,  
  "lat": 12.0,  
  "lon": 77.0  
}
```

Expected Result:

- Status Code: 400 Bad Request
 - Response Body: validation error
 - No DB entry created
-

Case 3: Custom List Action

- Method: GET
 - URL: http://127.0.0.1:8000/api/stations/all_names/
-

Case 4: Custom Detail Action

- Method: GET
 - URL: <http://127.0.0.1:8000/api/stations/1/ping/>
-

How to run the tests

- Ensure server is running:

`python manage.py runserver`

- Open:

`http://127.0.0.1:8000/api/stations/`

- Use browser form for POST
 - Use URLs for GET
-

This structure allows you to keep your IoT sensor data logic separate from your blog app while using a modern, professional API pattern.]